Занятие **46.1**. Указатели и const

На этом уроке мы рассмотрим указатели на константные переменные, указатели на константные значения, константные указатели и константные указатели на константные значения в языке C++.

Оглавление:

1. Указатели на константные переменные
2. Указатели на константные значения
3. Константные указатели
4. Константные указатели на константные значения
5. Заключение

Указатели на константные переменные

До этого момента все **указатели**, которые мы рассматривали, были неконстантными указателями на неконстантные значения:

```
int value = 7;
int *ptr = &value;
*ptr = 8; // изменяем значение value на 8
```

Однако, что произойдет, если указатель будет указывать на **константную** переменную?

```
const int value = 7; // value - это константа
int *ptr = &value; // ошибка компиляции: невозможно конвертировать const int* в int*
*ptr = 8; // изменяем значение value на 8
```

Фрагмент кода, приведенный выше, не скомпилируется: мы не можем присвоить неконстантному указателю константную переменную. Здесь есть смысл, ведь на то она и константа, что её значение нельзя изменить. Гипотетически, если бы мы могли присвоить константное значение неконстантному указателю, то тогда мы могли бы разыменовать неконстантный указатель и изменить значение этой же константы. А это уже является нарушением самого понятия «константа».

Указатели на константные значения

Указатель на константное значение — это неконстантный указатель, который указывает на неизменное значение. Для объявления указателя на константное значение, используется **ключевое слово const перед типом данных**:

```
const int value = 7;
const int *ptr = &value; // здесь всё ок: ptr - это неконстантный указатель, который указывает на "const int"
*ptr = 8; // нельзя, мы не можем изменить константное значение
```

В примере, приведенном выше, `ptr` указывает на константный целочисленный тип данных.

Пока что всё хорошо. Рассмотрим следующий пример:

```
int value = 7; // value - это не константа
const int *ptr = &value; // всё хорошо
```

Указатель на константную переменную может указывать и на неконстантную переменную (как в случае с переменной `value` в примере, приведенном выше). Подумайте об этом так: указатель на константную переменную обрабатывает переменную как константу при получении доступа к ней независимо от того, была ли эта переменная изначально определена как const или нет. Таким образом, следующее в порядке вещей:

```
int value = 7;
const int *ptr = &value; // ptr указывает на "const int"
value = 8; // переменная value уже не константа, если к ней получают доступ через неконстантный идентификатор
```

Но не следующее:

```
int value = 7;
const int *ptr = &value; // ptr указывает на "const int"
*ptr = 8; // ptr обрабатывает value как константу, поэтому изменение значения переменной value через ptr не допускается
```

Указателю на константное значение, который сам при этом не является константным (он просто указывает на константное значение), можно присвоить и другое значение:

```
int value1 = 7;
const int *ptr = &value1; // ptr указывает на const int

int value2 = 8;
ptr = &value2; // хорошо, ptr теперь указывает на другой const int
```

Константные указатели

Мы также можем сделать указатель константным. **Константный указатель** — это указатель, значение которого не может быть изменено после инициализации. Для объявления константного указателя используется **ключевое слово const между звёздочкой и именем указателя**:

```
int value = 7;
int *const ptr = &value;
```

Подобно обычным константным переменным, константный указатель должен быть инициализирован значением при объявлении. Это означает, что он всегда будет указывать на один и тот же адрес. В вышеприведенном примере `ptr` всегда будет указывать на адрес `value` (до тех пор, пока указатель не выйдет из **области видимости** и не уничтожится):

```
int value1 = 7;
int value2 = 8;

int * const ptr = &value1; // ок: константный указатель инициализирован адресом value1
ptr = &value2; // не ок: после инициализации константный указатель не может быть изменен
```

Однако, поскольку переменная `value`, на которую указывает указатель, не является константой, то её значение можно изменить путем разыменования константного указателя:

```
int value = 7;
int *const ptr = &value; // ptr всегда будет указывать на value
*ptr = 8; // ок, так как ptr указывает на тип данных (неконстантный int)
```

Константные указатели на константные значения

Наконец, можно объявить константный указатель на константное значение, используя **ключевое слово const как перед типом данных, так и перед именем указателя**:

```
int value = 7;
const int *const ptr = &value;
```

Константный указатель на константное значение нельзя перенаправить указывать на другое значение также, как и значение, на которое он указывает, — нельзя изменить.

Заключение

Подводя итоги, вам нужно запомнить всего лишь 4 правила:

- Неконстантный указатель можно перенаправить указывать на любой другой адрес.
- С помощью указателя на неконстантное значение можно изменить это же значение (на которое он указывает).
- Константный указатель всегда указывает на один и тот же адрес, и этот адрес не может быть изменен.
- Указатель на константное значение обрабатывает значение как константное (даже если оно таковым не является) и, следовательно, это значение через указатель изменить нельзя.

А вот с синтаксисом может быть немного труднее. Просто помните, что тип значения, на который указывает указатель, всегда находится слева (в самом начале):

```
int value = 7;
const int *ptr1 = &value; // ptr1 указывает на "const int", поэтому это указатель на константное значение
int *const ptr2 = &value; // ptr2 указывает на "int", поэтому это константный указатель на неконстантное значение
const int *const ptr3 = &value; // ptr3 указывает на "const int", поэтому это константный указатель на константное значение
```

Указатели на константные значения в основном используются в параметрах функций (например, при передаче массива) для гарантии того, что функция случайно не изменит значение(я) переданного ей аргумента.